

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Сортировка распределением (карманная сортировка) в деке на
базе связного списка. Поиск подстрок методом Бойера-Мура-Хорспула»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250

(подпись)

Проверил:

Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АЛГОРИТМ СОРТИРОВКИ.....	4
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ.....	5
3. БЛОК-СХЕМА АЛГОРИТМА.....	7
4. ТЕСТИРОВАНИЕ АЛГОРИТМА.....	7
5. ИСХОДНЫЙ КОД АЛГОРИТМА.....	8
ЗАКЛЮЧЕНИЕ.....	13
ПРИЛОЖЕНИЕ А.....	14

ВВЕДЕНИЕ

Цель работы — разработать программу сортировки распределением (карманной сортировки) последовательности вещественных чисел, которая будет храниться в деке, построенном на базе связного списка, а также предусмотреть возможность поиска подстроки в файле методом Бойера-Мура-Хорспула.

1. АЛГОРИТМ СОРТИРОВКИ И ПОИСКА

Программа получает на вход название файла, в котором хранятся исходные данные. Содержимое файла считывается целиком в динамический текстовый буфер (строку). Из этой строки последовательно считываются и преобразуются вещественные числа, которые затем записываются в двусвязный дек.

Далее происходит сортировка дека по следующему алгоритму:

1. Создаются карманы (тоже отдельные деки), количество карманов равно количеству элементов в деке;

2. Далее элементы распределяются по карманам. Для начала определяются максимальный (*max*) и минимальный (*min*) элементы дека. Индекс нужного кармана для каждого элемента определяется по формуле:

$$ind = \frac{(val - min) \cdot (numBuckets - 1)}{max - min}$$

3. Каждый заполненный карман сортируется на месте с помощью сортировки вставками;

4. Содержимое карманов последовательно сливается в один дек путем перемещения элементов.

Для поиска подстроки файл воспринимается как единая строка. Поиск подстроки выполняется методом Бойера-Мура-Хорспула. Алгоритм состоит из следующих этапов:

1. Строится таблица сдвигов для каждого символа алфавита на основе искомого шаблона. Величина сдвига для символа на позиции *i* шаблона длины *m* рассчитывается как *m - 1 - i*. Для символов, не входящих в шаблон, сдвиг равен *m*.

2. Поиск шаблона в тексте выполняется справа налево. При обнаружении несовпадения текущий символ текста используется для определения величины сдвига шаблона по таблице сдвигов.

2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Таблица 1 — Используемые переменные

Название	Тип	Границы типа	Назначение
value	float	от -3,4E+38 до 3,4E+38	Значение узла дека
next	struct Node*	-	Указатель на следующий узел дека
head	struct Node*	-	Указатель на начальный узел дека
tail	struct Node*	-	Указатель на конечный узел дека
size	size_t	от 0 до $2^{64} - 1$	Размер дека
node	struct Node*	-	Указатель на узел дека
oldHead	struct Node*	-	Указатель на удаляемый сначала узел
oldTail	struct Node*	-	Указатель на удаляемый с конца узел
cur	struct Node*	-	Указатель на текущий узел
next	struct Node*	-	Указатель на следующий узел в сортировке вставками
prev	struct Node*	-	Указатель на предыдущий узел в сортировке вставками
min	float	от -3,4E+38 до 3,4E+38	Минимальный элемент дека
max	float	от -3,4E+38 до 3,4E+38	Максимальный элемент дека
numBuckets	int	от 0 до 2147483647	Количество карманов
buckets	struct Deque*	-	Указатель на выделенную память для карманов
ind	int	от 0 до 2147483647	Индекс кармана для элемента дека
val	float	от -3,4E+38 до 3,4E+38	Значение удалённого элемента из дека
shift	int[256]	-	Таблица сдвигов для алгоритма Бойера-Мура-Хорспула
pattern	char[256]	-	Искомая подстрока (шаблон)

fileContent	char*	адрес памяти	Буфер для хранения содержимого файла в виде строки
pos	int	от -1 до 2147483647	Индекс начала найденной подстроки в тексте
file	FILE*	-	Указатель на файловый поток для чтения
tmp	float	от -3,4E+38 до 3,4E+38	Переменная для хранения прочитанного числа
argc	int	от 1 до 2147483647	Хранит количество аргументов при запуске программы
argv[]	char**	адрес памяти	Массив переданных аргументов
filename	char*	адрес памяти	Ссылается на имя файла
d	struct Deque*	-	Локальная переменная для дека
i	int	от 0 до 2147483647	Переменная для итерации циклов

3. БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

4. ТЕСТИРОВАНИЕ АЛГОРИТМА

```
ЛРЗ > ./lab3 data.txt
Исходный текст файла:
3 43 4325 1 46 12.44 -123.34 21947839 300 1 -200 -100 200 1000

Lorem ipsum dolor sit amet consectetur adipisicing elit. Voluptas temporibus
dolores animi obcaecati omnis aspernatur ratione nam ipsa, mollitia, unde
reiciendis. Magni nesciunt eos dignissimos quidem dolore iure assumenda saepe.

--- Исходные данные дека ---
[ 3.000000 <-> 43.000000 <-> 4325.000000 <-> 1.000000 <-> 46.000000 <-> 12.440000 <-> -123.339996 <->
21947840.000000 <-> 300.000000 <-> 1.000000 <-> -200.000000 <-> -100.000000 <-> 200.000000 <-> 1
000.000000 ] (size=14)
Введите подстроку для поиска методом Бойера-Мура-Хорспула: nam
Поиск подстроки "nam"...
Подстрока найдена на позиции: 190

--- Отсортированные данные дека ---
[ -200.000000 <-> -123.339996 <-> -100.000000 <-> 1.000000 <-> 1.000000 <-> 3.000000 <-> 12.440000
<-> 43.000000 <-> 46.000000 <-> 200.000000 <-> 300.000000 <-> 1000.000000 <-> 4325.000000 <-> 21947
840.000000 ] (size=14)
```

Рисунок 1 — Результат успешной работы алгоритма

```
ЛРЗ > ./lab3
Использование: ./lab3 <имя_файла> [подстрока]
ЛРЗ > ./lab3 data1.txt
Ошибка: не удалось открыть файл data1.txt
```

Рисунок 2 — Результат работы алгоритма при неуказанном входном файле или ошибке при открытии файла

5. ИСХОДНЫЙ КОД АЛГОРИТМА

Программа написана на языке C, для компиляции использовался компилятор gcc.

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
3 | TARGET = lab3
4 |
5 | all: $(TARGET)
6 |
7 | $(TARGET): lab3.c
8 |     $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET)
9 |
10 | clean:
11 |     rm -f $(TARGET)
```

Исходный код lab3.c:

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include "stddef.h"
5 | #include "stdbool.h"
6 |
7 | #define ALPHABET_SIZE 256
8 |
9 | typedef struct Node {
10 |     float value;
11 |     struct Node *next;
12 |     struct Node *prev;
13 | } Node;
14 |
15 | typedef struct Deque {
16 |     struct Node *head;
17 |     struct Node *tail;
18 |     size_t size;
19 | } Deque;
20 |
21 | void init(Deque *d) {
22 |     d->head = d->tail = NULL;
23 |     d->size = 0;
24 | }
25 |
26 | void pushFront(Deque *d, float value) {
27 |     Node *node = malloc(sizeof(Node));
28 |     if (!node) {
29 |         fprintf(stderr, "Ошибка: не удалось выделить память для узла дека.\n");
30 |         exit(1);
31 |     }
32 |     node->value = value;
33 |     node->next = d->head;
34 |     node->prev = NULL;
35 |
36 |     if (d->size == 0) {
37 |         d->head = node;
38 |         d->tail = node;
39 |     } else {
40 |         d->head->prev = node;
41 |         d->head = node;
42 |     }
43 |     d->size++;
44 | }
45 |
46 | void pushBack(Deque *d, float value) {
```



```

47     Node *node = malloc(sizeof(Node));
48     if (!node) {
49         fprintf(stderr, "Ошибка: не удалось выделить память для узла дека.\n");
50         exit(1);
51     }
52     node->value = value;
53     node->next = NULL;
54     node->prev = d->tail;
55
56     if (d->size == 0) {
57         d->head = node;
58         d->tail = node;
59     } else {
60         d->tail->next = node;
61         d->tail = node;
62     }
63     d->size++;
64 }
65
66 bool popFront(Deque *d, float *out) {
67     if (d->size == 0) return false;
68     Node *oldHead = d->head;
69     *out = oldHead->value;
70
71     d->head = oldHead->next;
72     if (d->head == NULL) {
73         d->tail = NULL;
74     } else {
75         d->head->prev = NULL;
76     }
77
78     free(oldHead);
79     d->size--;
80     return true;
81 }
82
83 bool popBack(Deque *d, float *out) {
84     if (d->size == 0) return false;
85     Node *oldTail = d->tail;
86     *out = oldTail->value;
87
88     d->tail = oldTail->prev;
89     if (d->tail == NULL) {
90         d->head = NULL;
91     } else {
92         d->tail->next = NULL;
93     }
94
95     free(oldTail);
96     d->size--;
97     return true;
98 }
99
100 void printD(Deque *d) {
101     if (d->size == 0) {
102         printf("[ пусто ]\n");
103         return;
104     }
105
106     Node *cur = d->head;
107     printf("[ ");
108     for (size_t i = 0; i < d->size; i++) {
109         printf("%f ", cur->value);
110         if (i + 1 < d->size) printf("<-> ");
111         cur = cur->next;
112     }
113     printf(" ] (size=%zu)\n", d->size);
114 }
115
116 void insertionSort(Deque *d) {
117     size_t n = d->size;

```

```

118     if (n <= 1) return;
119
120     Node *cur = d->head->next;
121     while (cur != NULL) {
122         Node *next = cur->next;
123         float val = cur->value;
124         Node *prev = cur->prev;
125
126         while (prev != NULL && prev->value > val) {
127             prev = prev->prev;
128         }
129
130         if (cur->prev != NULL) {
131             cur->prev->next = cur->next;
132         } else {
133             d->head = cur->next;
134         }
135
136         if (cur->next != NULL) {
137             cur->next->prev = cur->prev;
138         } else {
139             d->tail = cur->prev;
140         }
141
142         if (prev == NULL) {
143             cur->next = d->head;
144             cur->prev = NULL;
145             if (d->head != NULL) {
146                 d->head->prev = cur;
147             }
148             d->head = cur;
149         } else {
150             cur->next = prev->next;
151             cur->prev = prev;
152             if (prev->next != NULL) {
153                 prev->next->prev = cur;
154             } else {
155                 d->tail = cur;
156             }
157             prev->next = cur;
158         }
159         cur = next;
160     }
161 }
162
163 void bucketSort(Deque *d) {
164     size_t n = d->size;
165     if (n <= 1) return;
166
167     float min = d->head->value, max = d->head->value;
168     Node *cur = d->head->next;
169     for (size_t i = 1; i < n; i++) {
170         if (cur->value < min) min = cur->value;
171         if (cur->value > max) max = cur->value;
172         cur = cur->next;
173     }
174
175     if (min == max) return;
176
177     int numBuckets = (int)n;
178     Deque *buckets = malloc(numBuckets * sizeof(Deque));
179     if (!buckets) {
180         fprintf(stderr, "Ошибка: не удалось выделить память для корзин\n"); exit(1);
181     }
182     for (int i = 0; i < numBuckets; i++)
183         init(&buckets[i]);
184
185     while (d->size != 0) {
186         float val;
187         popFront(d, &val);
188         int ind = (int)((val - min) * (numBuckets - 1) / (max - min));

```

```

189         pushBack(&buckets[ind], val);
190     }
191
192     for (int i = 0; i < numBuckets; i++) {
193         if (buckets[i].size == 0) continue;
194         insertionSort(&buckets[i]);
195         float val;
196         while (buckets[i].size != 0) {
197             if (popFront(&buckets[i], &val)) {
198                 pushBack(d, val);
199             }
200         }
201     }
202
203     free(buckets);
204 }
205
206 void buildShiftTable(const char *pattern, size_t m, int shift[ALPHABET_SIZE]) {
207     for (int i = 0; i < ALPHABET_SIZE; i++) {
208         shift[i] = (int)m;
209     }
210     for (size_t i = 0; i < m - 1; i++) {
211         shift[(unsigned char)pattern[i]] = (int)(m - 1 - i);
212     }
213 }
214
215 int bmhSearch(const char *text, const char *pattern) {
216     size_t n = strlen(text);
217     size_t m = strlen(pattern);
218     if (m == 0 || m > n) return -1;
219
220     int shift[ALPHABET_SIZE];
221     buildShiftTable(pattern, m, shift);
222
223     size_t i = m - 1;
224     while (i < n) {
225         size_t k = 0;
226         while (k < m && pattern[m - 1 - k] == text[i - k]) {
227             k++;
228         }
229         if (k == m) {
230             return (int)(i - m + 1);
231         }
232         i += shift[(unsigned char)text[i]];
233     }
234     return -1;
235 }
236
237 char* readFileToString(const char *filename) {
238     FILE *file = fopen(filename, "r");
239     if (!file) {
240         fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", filename);
241         exit(1);
242     }
243
244     fseek(file, 0, SEEK_END);
245     long length = ftell(file);
246     fseek(file, 0, SEEK_SET);
247
248     char *buffer = malloc(length + 1);
249     if (!buffer) {
250         fclose(file);
251         fprintf(stderr, "Ошибка: не удалось выделить память для буфера файла\n");
252         exit(1);
253     }
254
255     size_t read_bytes = fread(buffer, 1, length, file);
256     buffer[read_bytes] = '\0';
257     fclose(file);
258     return buffer;
259 }

```

```

260
261 void parseBufferToDeque(const char *buffer, Deque *d) {
262     const char *ptr = buffer;
263     float tmp;
264     int offset;
265     while (sscanf(ptr, "%f%n", &tmp, &offset) == 1) {
266         pushBack(d, tmp);
267         ptr += offset;
268     }
269 }
270
271 int main(int argc, char **argv) {
272     if (argc < 2) {
273         fprintf(stderr, "Использование: %s <имя_файла> [подстрока]\n", argv[0]);
274         return 1;
275     }
276
277     char *filename = argv[1];
278     char *fileContent = readFileToString(filename);
279
280     Deque d;
281     init(&d);
282     parseBufferToDeque(fileContent, &d);
283
284     printf("Исходный текст файла:\n%s\n", fileContent);
285     printf("--- Исходные данные дека ---\n");
286     printD(&d);
287
288     char pattern[256];
289     if (argc >= 3) {
290         strncpy(pattern, argv[2], sizeof(pattern) - 1);
291         pattern[sizeof(pattern) - 1] = '\0';
292     } else {
293         printf("Введите подстроку для поиска методом Бойера-Мура-Хорспула: ");
294         if (fgets(pattern, sizeof(pattern), stdin)) {
295             pattern[strcspn(pattern, "\n")] = '\0';
296         }
297     }
298
299     printf("Поиск подстроки \"%s\"...\n", pattern);
300     int pos = bmhSearch(fileContent, pattern);
301     if (pos != -1) {
302         printf("Подстрока найдена на позиции: %d\n", pos);
303     } else {
304         printf("Подстрока не найдена.\n");
305     }
306
307     bucketSort(&d);
308     printf("\n--- Отсортированные данные дека ---\n");
309     printD(&d);
310
311     free(fileContent);
312     float trash;
313     while (d.size > 0) {
314         popFront(&d, &trash);
315     }
316
317     return 0;
318 }

```

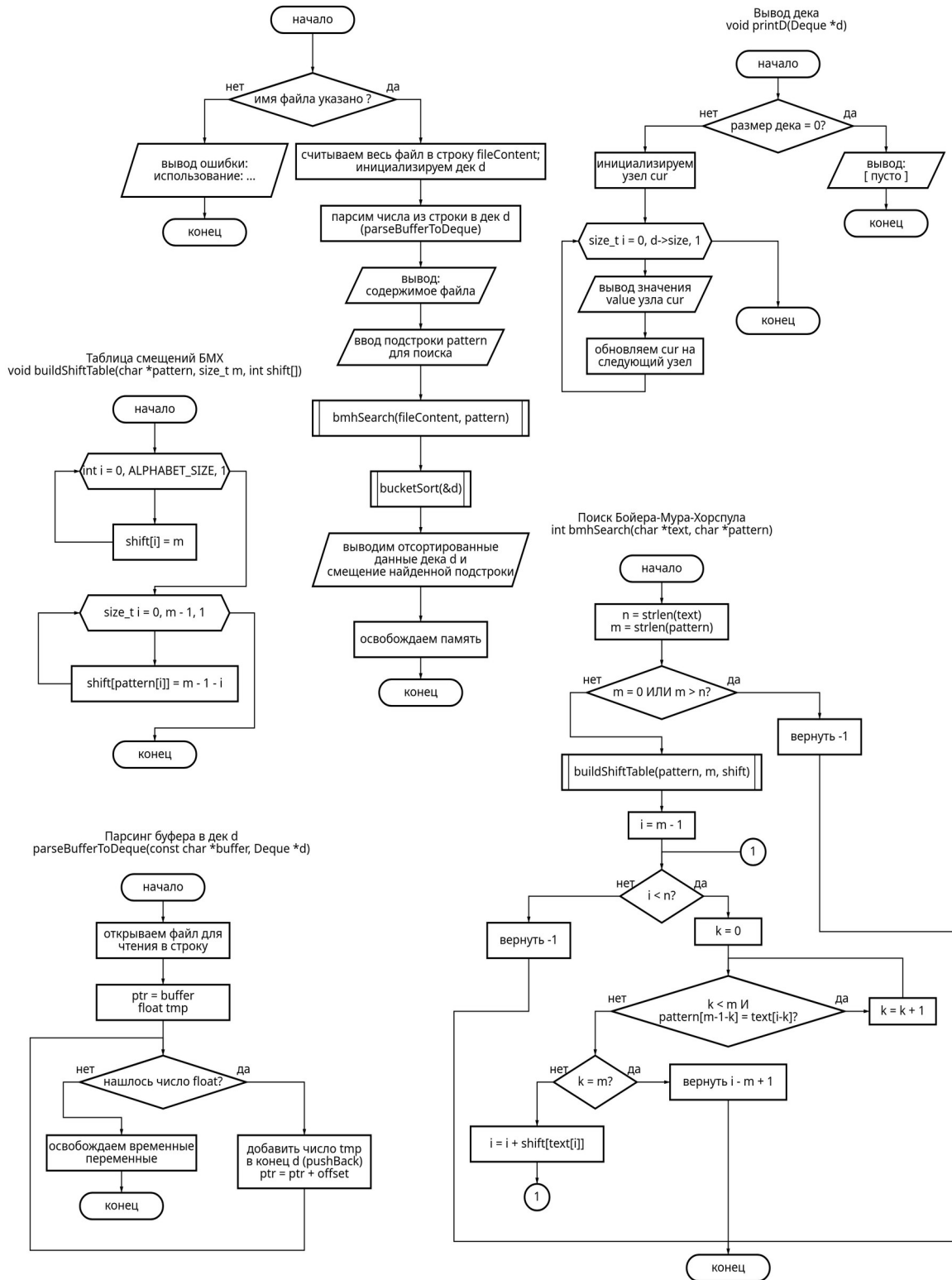
ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана программа на языке C для сортировки последовательности чисел с помощью алгоритма распределения (карманная сортировка), использующего дек, построенного на базе связного списка. Для решения задачи также был реализован алгоритм сортировки вставками. Числа считывались из файла и хранились в деке. Помимо этого был реализован поиск подстроки в файле методом Бойера-Мура-Хорспула.

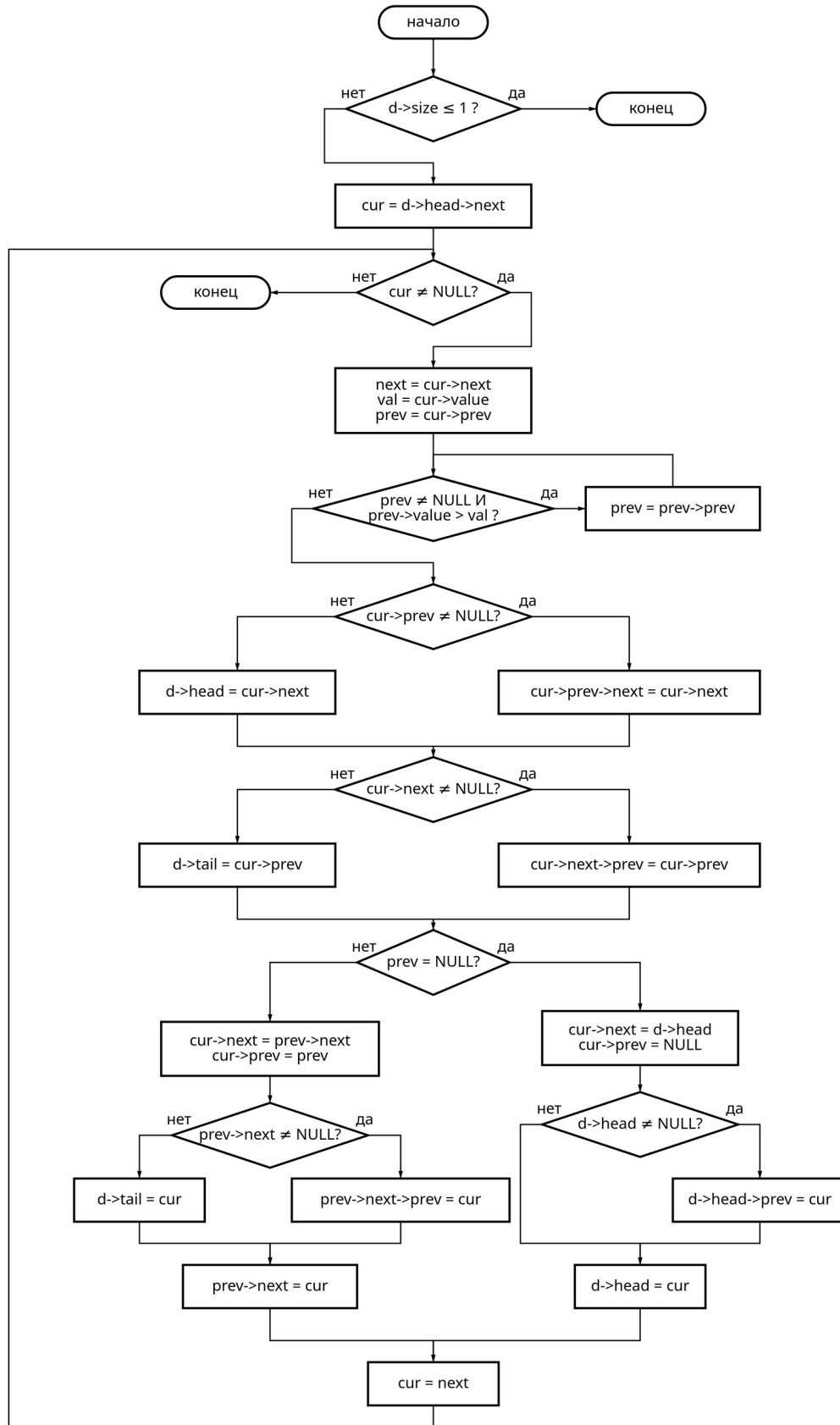
Программа разрабатывалась в среде VS Code, компилировалась с помощью gcc и запускалась в ОС Arch Linux.

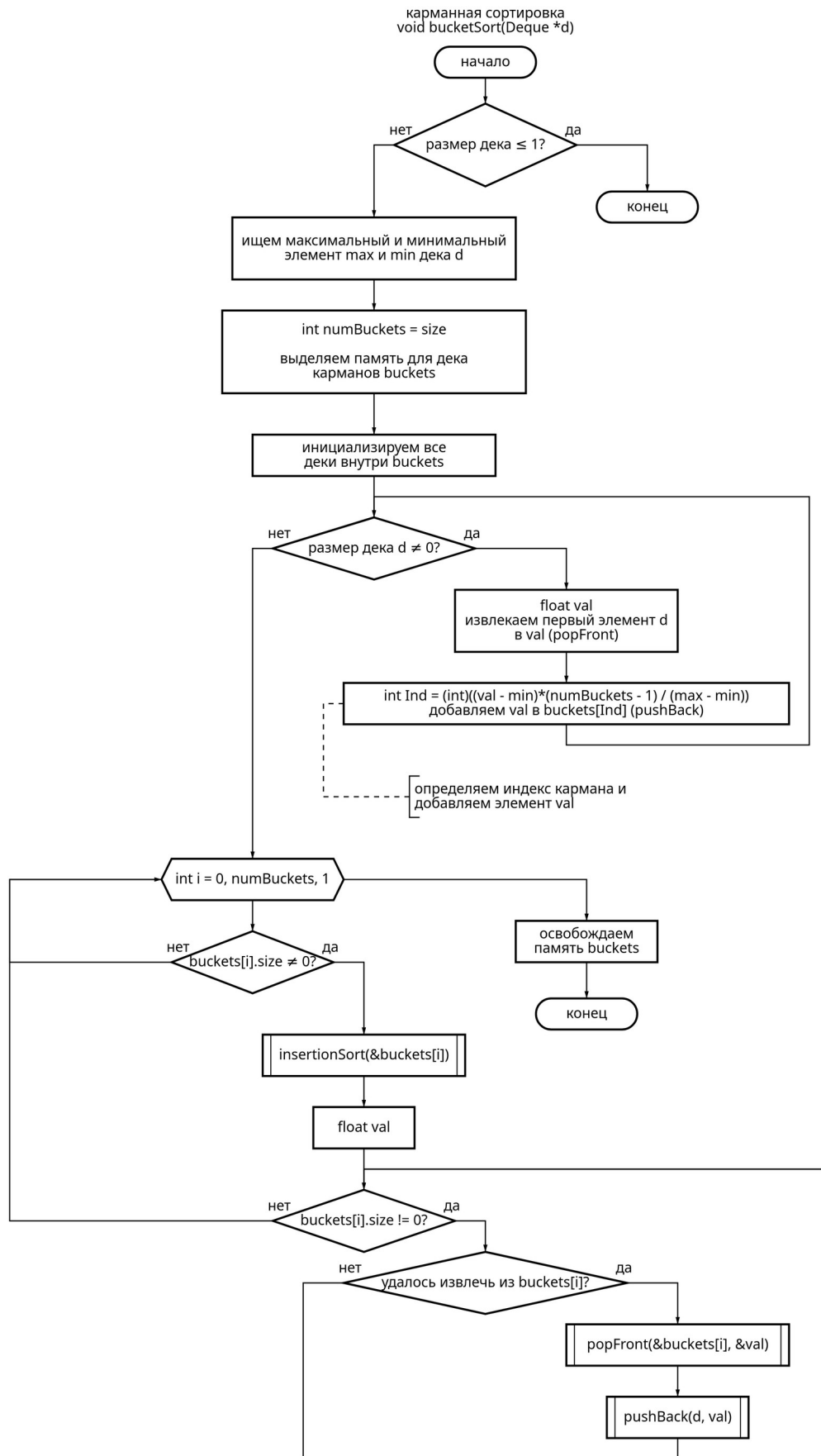
Тестирование алгоритма показало его корректную работу при различных входных данных. Выполнение работы позволило закрепить навыки разработки алгоритмов сортировки и поиска подстрок, а также углубить знания в области структур данных.

ПРИЛОЖЕНИЕ А

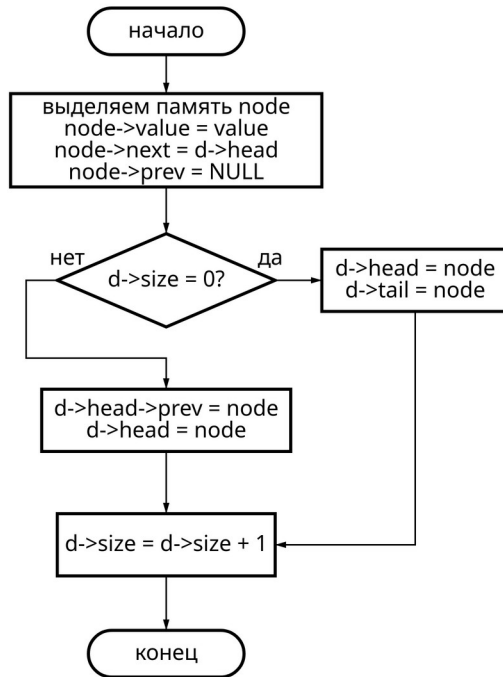


Сортировка вставками
void insertionSort(Deque *d)

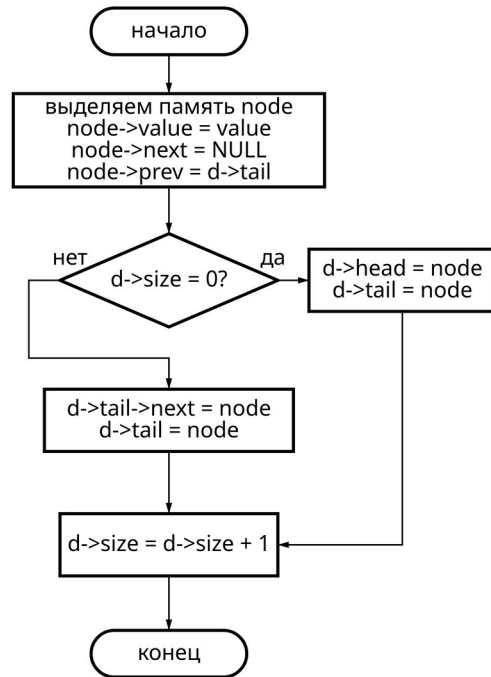




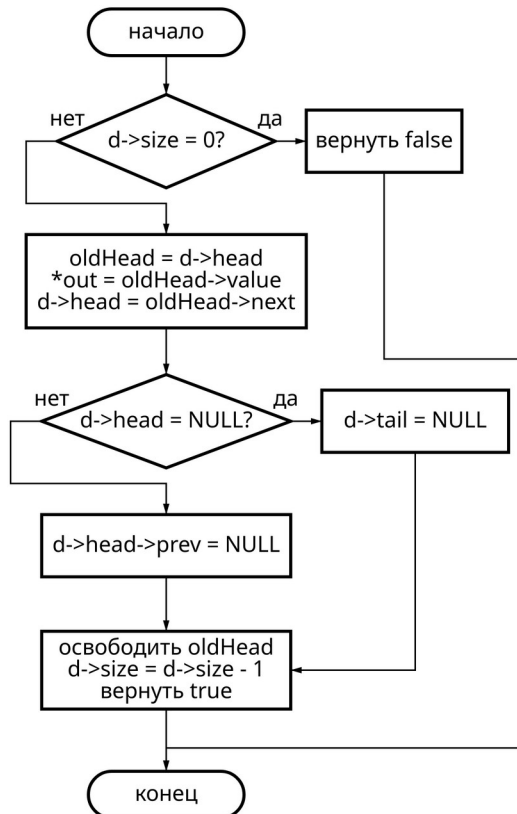
Вставка в начало
void pushFront(Deque *d, float value)



Вставка в конец
void pushBack(Deque *d, float value)



Удаление из начала
bool popFront(Deque *d, float *out)



Удаление из конца
bool popBack(Deque *d, float *out)

